Chapter 16 worked solutions

From Pretrained Weights to Task Behavior

This guide covers Exercises 1–4 and 10–11. It counts which weights an adaptation trains, aligns response targets with the logits that predict them, and works through a low-rank update and a preference loss. The companion notebook covers Exercises 5–9 and is also available as a PDF.

1. Count a task head

Think of an encoder as producing a vector of six numbers for each request. The new classification head uses that vector to produce three **logits**, one score per class before softmax. If the feature vector is $\mathbf{h} \in \mathbb{R}^6$, the head computes

$$\mathbf{z} = W\mathbf{h} + \mathbf{b}, \qquad W \in \mathbb{R}^{3\times 6}, \qquad \mathbf{b} \in \mathbb{R}^3.$$

Each of the three rows of W supplies the six weights for one class score. The bias vector adds one separately learned offset to each score. This gives $3 \cdot 6 = 18$ weights and three biases, for **21 head parameters**. Softmax, if used to turn the logits into probabilities, introduces no trainable parameters.

Training setup	Trainable encoder parameters	Trainable head parameters	Total trainable
Frozen encoder, train the head	0	21	**21**
Full fine-tuning, train encoder and head	120	21	**141**

The encoder still computes the six features in the frozen run. Its 120 parameters still belong to the model, but the optimizer updates only the head. Both setups therefore contain 141 parameters in total. The difference is which of those parameters can change during training.

2. Mark the response targets

The packed sequence begins with three prompt tokens, **BOS**, **Question**, and **Assistant**. The response is **New**, **Address**, and **EOS**. **PAD** fills the final slot for batching. To align each next-token target with the logit row that predicts it, put token j from the packed sequence under input position $j - 1$. This produces two rows of length six. The input row omits the final packed token and the target row omits the first.

Shifted column, zero-based	0	1	2	3	4	5
Input position and logit row	BOS	Question	Assistant	New	Address	EOS
Next-token target before masking	Question	Assistant	New	Address	EOS	PAD
Target loss weight	0	0	**1**	**1**	**1**	0
Label used for loss	Ignore	Ignore	**New**	**Address**	**EOS**	Ignore

In the original, unshifted layout the response mask is $(0, 0, 0, 1, 1, 1, 0)$. Drop its first entry along with the first target token to obtain $(0, 0, 1, 1, 1, 0)$ in the aligned layout. The first supervised token is **New** at packed index 3, so **its predictor is the logit row at Assistant, index 2**. Those logits are computed after the model has read the prompt and before it has seen **New**. The logit row at **New** is instead trained to predict **Address**.

EOS has weight one because ending the response is part of the behavior to learn. **PAD** has weight zero because it is a batching placeholder. The zero weights on the prompt targets only remove those predictions from the loss. The prompt tokens remain available as context under the causal attention mask. In particular, the input token **Assistant** is part of the prompt even though the next-token prediction from its row is scored.

3. Follow a rank-one update

LoRA keeps the base projection W_0 and adds a smaller trainable path. In this example A has shape 1×2, so it reduces a two-coordinate input to one scalar. B has shape 2×1, so it turns that scalar back into two output coordinates. The branch scale is $\alpha/r = 0.5/1 = 1/2$.

The base is the identity, so it returns the input. The adapter first computes $A\mathbf{x} = 2 \cdot 1 + 1 \cdot 2 = 4$. Applying B to 4 and then scaling gives the second branch.

$$W_0\mathbf{x} = \begin{bmatrix} 1 \\ 2 \end{bmatrix}, \qquad \frac{\alpha}{r}B(A\mathbf{x}) = \frac{1}{2}\begin{bmatrix} 1 \\ -1 \end{bmatrix}4 = \begin{bmatrix} 2 \\ -2 \end{bmatrix}.$$

The two branches add coordinate by coordinate, giving $(3, 0)^\top$. The update can only move the output in the direction $(1, -1)^\top$ because B has a single column. The scalar read $A\mathbf{x}$ controls how far it moves along that direction.

For the merged calculation, multiply B and A in that order. Their product has the same 2×2 shape as W_0.

$$BA = \begin{bmatrix} 2 & 1 \\ -2 & -1 \end{bmatrix}, \qquad W_{merged} = W_0 + \frac{1}{2}BA = \begin{bmatrix} 2 & 1/2 \\ -1 & 1/2 \end{bmatrix}.$$

Multiplying by the original input gives

$$W_{merged}\mathbf{x} = \begin{bmatrix} 2 \cdot 1 + (1/2) \cdot 2 \\ -1 \cdot 1 + (1/2) \cdot 2 \end{bmatrix} = \begin{bmatrix} 3 \\ 0 \end{bmatrix}.$$

The merged result agrees because matrix multiplication distributes over addition and $B(A\mathbf{x}) = (BA)\mathbf{x}$. The exercise has no bias. If a base bias were present, the merged projection would add the same bias once after applying the merged weight.

4. Compute a relative preference

A chosen response and a rejected response have one summed log-probability each. A model's **gap** is its chosen score minus its rejected score. This sign convention makes the gap positive when the model scores the chosen response higher. The two gaps are

$$g_{policy} = -2 - (-3.5) = 1.5, \qquad g_{ref} = -3 - (-3.25) = 0.25.$$

The reference already favors the chosen response by 0.25. The policy's preference is stronger by $1.5 - 0.25 = 1.25$. DPO subtracts the reference gap and scales by β, so the margin at $\beta = 1$ is $m = 1.25$. With sigmoid $\sigma(m) = 1/(1 + e^{-m})$, the loss for this pair is

$$\ell = -\log \sigma(1.25) = \log\left(1 + e^{-1.25}\right) \approx 0.251929.$$

Subtracting 0.5 from both policy scores gives $(-2.5, -4)$. The policy gap is still $-2.5-(-4)=1.5$. The reference gap remains 0.25, the margin remains 1.25, and the loss is **unchanged**.

More generally, replacing the policy scores $(p_{chosen}, p_{rejected})$ with $(p_{chosen}-c, p_{rejected}-c)$ cancels c in their difference. The two response probabilities have both been multiplied by e^{-c}, leaving their ratio unchanged. This single-pair loss depends on that relative preference, measured against the reference, so it cannot distinguish the two sets of scores in this exercise.

10. Plan an adaptation comparison (optional)

The message *Please tell me the address on file* asks to look up an address. The message *Please use my new address* asks to update it. Use the labels **LOOKUP** and **UPDATE** for the two intents. Give every system the same interface. A request goes in, a two-class head returns one logit per label, and the larger logit determines the answer. Use the same encoder output vector for the head, such as the representation at a designated classification position. This keeps the output and loss comparable across the three ways of adapting the model.

Start all three systems from the same identified encoder revision and tokenizer. Give them the same architecture and initial values for the head. For each of several seeds, make fresh copies of that initial state. The only intended change is where training can update weights.

Method	What the optimizer can update
Frozen encoder	The weights and bias of the new head
Full fine-tuning	All encoder parameters and the head
Low-rank adaptation	The head and rank-four LoRA factors on the query and value projections of each encoder layer. Keep the pretrained projections fixed.

Set the adapter scale parameter to eight and start its second factor at zero. The adapter run and the frozen run then begin with the same encoder function. Use the same input formatting, truncation limit, tokenization, batch order, cross-entropy objective, and update budget. Fix dropout behavior across the comparison. Use the same optimizer family and give each method the same number of predeclared learning-rate trials on validation data. The best learning rate can differ because training all weights and training only a small adapter change the function at different rates. Select each method's checkpoint by the same validation metric.

Group repeated and paraphrased templates before making the splits. For instance, *Please change my address to [ADDRESS]* and lightly edited variants belong to one template family. Putting one version in training and another in testing could reward recognition of the wording. Assign entire families to training, validation, or testing. If messages come from threads or accounts, keep linked examples together as well. Verify that each split contains both intents. Include difficult pairs such as *Can you confirm the new address on file?* and *Can you put my new address on file?* so that the phrase *new address* appears with both labels. Reserve the test set until the hyperparameters and checkpoint rules have been chosen.

Report exact-match accuracy and the two-by-two confusion matrix. Include the per-class recall, which is the fraction of actual requests of each class that the model identifies correctly. This shows whether a system handles frequent lookup requests well while missing updates. Break out performance on template families that do not occur in training. Compare runs on the same examples and report variation across seeds. If estimating sampling uncertainty, resample entire test families so closely related examples remain grouped.

Count trainable scalar parameters from the optimizer's parameter list. For encoder output width d, the two-class head has $2d + 2$ trainable scalars. Full fine-tuning adds the parameter count of the encoder. A rank-r adapter for a projection from d_{in} to d_{out} adds $r(d_{in} + d_{out})$ scalars. Sum this over the selected query and value projections and add the head. Report the total deployed parameter count as well as the number trained. Say whether the LoRA factors stay separate or are merged into the base matrices, since merging removes the separate factor tensors from the deployed model.

Report two storage figures using the same saved dtype. The **task-specific checkpoint** contains the head for frozen adaptation, the factors and head for LoRA, and the updated encoder and head for full fine-tuning. The **deployed footprint** also includes the base wherever it is needed. Report raw tensor payloads or actual checkpoint file sizes and say which one you use. File headers, compression, and shared base weights affect the relationship between those numbers. Record training-only optimizer state separately if measuring it. The comparison can then show whether updating fewer parameters changes performance on new request wording and how much task-specific storage it requires.

11. Rebuild an adapted model (optional)

Suppose a colleague has the base encoder and wants to reproduce the adapted classifier. Save enough information to reconstruct the adapted function and to verify which base it modifies. A query matrix and a value matrix can have the same shape. The matrix shape alone cannot tell the loader which one an adapter belongs to.

Save	Why it matters
Immutable base revision, architecture configuration, and weight fingerprint	The same public model name can refer to different weights.
Tokenizer revision, input formatting, truncation, and pooling rule	The same string must produce the same input IDs and head representation.
The learned A and B arrays for each adapted projection	The correction is built from their product.
Full module path and layer number for each pair, input and output dimensions, rank r, scale parameter α, matrix orientation, and any adapter dropout setting	These determine where the factors attach and how the layer computes its correction.
Classifier weight, bias, input width, and class order	The head converts the shared representation to the saved class logits.
Tensor dtype and checkpoint format version	The loader can check that it is restoring a compatible state.

For a base layer W_0 in the chapter's convention, restore the factor shapes $A \in \mathbb{R}^{r \times d_{in}}$ and $B \in \mathbb{R}^{d_{out} \times r}$. The applied weight is $W_0 + (\alpha/r)BA$. Keep the original bias unless a trained bias was explicitly part of the adaptation and is saved. The loader should resolve every recorded module path, check the dimensions and orientation, and check for missing or unexpected tensors. Verify the base fingerprint before loading the factors.

Before handing off the checkpoint, put the trained model in evaluation mode and save a small fixture. Use two or three requests with different lengths and intents, and record both the packed input IDs and the expected pre-softmax class logits. Select examples that exercise both adapted projections. Include a batch with padding if padding is part of the expected input. Save its attention mask and any token-type IDs that the encoder consumes. After reloading, compare the IDs exactly and the logit arrays within a stated tolerance. For example, on the same CPU backend in float32 use absolute tolerance 10^{-6} and relative tolerance 10^{-5}, then report the largest observed difference. Comparing class labels alone can miss an error that changes the logits without changing their order.

Also save a probe at one known adapted layer. Choose an input vector $\mathbf{x}$ for which the saved query adapter and value adapter have nonzero, distinguishable corrections. Feed that exact vector directly to each named projection with its adapter off and then on. Record both output differences so a change in upstream representations cannot alter the input to this probe. For a query projection, the expected difference between its adapted and base outputs is $(\alpha/r)B_Q A_Q \mathbf{x}$. Do the analogous check for the value projection using its own factors. Comparing both local differences helps catch adapters swapped between projections of the same shape. An incorrect base can still yield the right adapter difference because the base term cancels, so the base fingerprint and end-to-end logits are separate checks. Together the checks test the restored base, the placement of the adapters, and the classifier that uses their outputs.